Safe Reuse Is Not Enough: A Controlled Comparison of Semantic Evidence Persistence, Full Reverification, and Strengthened Memoization

Erik Fisher Cueva Huari
Independent Researcher, VentureSynaptic Labs, Peru
ORCID: 0000-0001-7268-5510

Draft manuscript — 1 November 2026
Not peer reviewed; not yet submitted

Abstract

Persisting formal evidence across software revisions promises to avoid repeated verification while preserving the validity of earlier results. Whether a semantically informed evidence store offers an advantage over ordinary query memoization, however, requires a matched comparison rather than a full-recomputation baseline alone. We report EXP-019, a pre-specified and internally audited controlled experiment on a deliberately bounded bit-vector domain. Three conditions process the same 64 revision transitions: full reverification (A), strengthened SMT-query memoization (B), and an implemented semantic evidence store (C). The primary outcome is actual solver invocations during revision processing, not elapsed time. Condition C uses 252 invocations versus 576 for A (56.25% fewer), but 96 for B (2.625 times as many). Eight adversarial controls passed for C; all 576 target obligations were decided and no confirmed invalid evidence reuse was observed in the tested domain. The store performed better than B only for the equivalent-change stratum (32 versus 40 invocations). A subsequent, explicitly exploratory hybrid prototype reduced some calls but increased wall-clock cost and exposed implementation defects. These results support bounded safe reuse and savings against full reverification, but do not support an overall efficiency advantage over a strong conventional incremental baseline. We provide the protocol, measurement boundary, audit trail, and limitations to make this negative result independently assessable. **Keywords:** incremental verification; SMT; evidence reuse; memoization; negative results; software evolution; reproducibility.

1 Introduction

Verification results are often sensitive to changes in program versions, definitions, dependencies, and toolchains. Reusing a result requires more than establishing that its conclusion happens to remain true: the evidence must still be applicable to the revised artifact. Evidence stores can track revision identity, causal dependencies and provenance, but this machinery also introduces costs. Moreover, standard caching can already avoid repeat SMT queries. The scientific question is thus comparative: what efficiency benefit, if any, is attributable to semantic evidence management beyond an appropriately strengthened cache?

EXP-019 is an experiment within the EFCH research program on property-oriented software engineering. It tests a concrete implementation rather than the broader paradigm or its language. Our contributions are (i) an explicit distinction between semantic truth and contractual evidence validity, (ii) a three-condition matched comparison with a strong query-cache baseline, (iii) an internally audited negative result with stratum-level analysis, and (iv) an account of subsequent exploratory checks without treating them as confirmatory evidence. We make no claim of a new verification algorithm or general superiority.

2 Background and Related Work

Incremental computation and memoization have a substantial history; dependency tracking and cached computation can avoid recomputing unchanged work [1], and build systems formalize the same trade-off between dependency tracking and rebuilding [4]. SMT solvers such as Z3 provide automated checking of formulas including finite-width bit-vectors [2]. Evaluation of software engineering interventions benefits from explicit questions, controlled alternatives and documented threats to validity [3].

Caching and reuse in verifiers. Reusing verification results across edits is established practice. Leino and Wüstholz cache Boogie verification results at the granularity of individual checks, using dependency information to decide what must be rechecked after an edit [5]. Proof environments such as Why3 store proof sessions and replay or mark them obsolete after changes [6], and Frama-C/WP maintains a proof cache with update and replay modes [7]. F* caches verified modules keyed on transitive hashes [8]. Beckert and Klebanov reuse proof steps across program versions in KeY [9]. Symbolic execution and program analysis tools reuse solver work through constraint caching and canonicalization: KLEE caches counterexamples and exploits constraint independence [10], and Green canonicalizes and per-

sists constraint results across runs and programs [11]. These systems show that both query-level caching and evidence-aware reuse have direct antecedents; EXP-019 does not claim either as new.

Verification of program changes. Regression verification proves equivalence between closely related versions [12], and differential verifiers such as SymDiff check relational properties between versions [13]. eVolCheck [14] and UpProver [15] reuse and repair function summaries to recheck upgrades incrementally. Incremental bounded model checking reuses solver state across unwindings [16], and incremental proof formats allow such runs to be certified [17]. Certificate-based approaches such as proof-carrying code [18] and SMTCoq [19] separate the producer of evidence from an independent checker, a guarantee that the store evaluated here does not provide (Section 6).

Position of this work. The question we address is therefore not whether evidence can be reused, but whether a semantic evidence store, as implemented, yields a measurable efficiency advantage over a conventional cache given the same inputs and solver. We are not aware of a pre-specified matched comparison of this kind, but our literature coverage is targeted rather than systematic, and we make no novelty claim for the mechanisms themselves.

Here, the distinction is operational. A query cache B recognizes equivalent serialized SMT queries under a fixed normalization and tool-base descriptor. The evidence store C additionally manages certificates and revision-aware applicability rules; it uses equality proofs and transfer rules where appropriate. Those additional guarantees should be assessed independently of their computational price.

3 Research Questions and Experimental Design

3.1 Research questions

We distinguish three pre-specified outcomes:

R1: safety.
Does C decide the required obligations without confirmed semantically incorrect or contractually stale evidence reuse, and pass eight targeted controls?

R2: savings over A.
Does C use fewer actual SMT solver invocations than full reverification?

R3: differential efficiency.
Does C outperform strengthened memoization B on the primary solver-call measure?

None of these logically implies another. R1 pertains to the bounded implementation and corpus, not a proof of general store correctness.

3.2 Domain and revisions

The evaluation uses a total, small bit-vector core with one input and output of width $w \in \{1, 2, 3, 4\}$. Expressions use literals, the input variable, addition, subtraction and function calls. Each initial state contains functions $h = x$, $q = 0$, and $f = B$, with three candidates for f: $c_0 = B$, $c_1 = 0 + B$, and $c_2 = (B + 1) - 1$. The body B is selected from identity, zero, increment and a call to h. Changes affect unrelated q or h, replace f with a constructed equivalent expression, replace it with a non-equivalent expression, or alter a dependency. The deterministic grid yields 64 transitions; eight additional controls make 72 evaluation entries. A separate 40-entry preparation collection was used for instrument development and calibration. The evaluation is a designed microbenchmark, not a representative sample of industrial repositories.

Each destination state S_d has nine target obligations: well-definedness for f, h, q and three candidates, plus three realization claims comparing the candidates with f. Hence the 64 transitions contain $64 \cdot 9 = 576$ target obligations. The claim that these nine are sufficient is restricted to this constructed model.

3.3 Conditions and parity

A—full reverification: independently checks the nine destination obligations without retaining verification results. **B—strengthened memoization:** executes the same destination procedure as A but wraps the original SMT proving routine with a cache of queries. Input variables are renamed positionally while preserving sorts; the query key includes normalized serialized assertions and a canonical descriptor of the tool base. It does not simplify or algebraically canonicalize expressions. Only decided proven/refuted outcomes are cached; counterexamples are remapped and reconfirmed. B does not perform inter-version equivalence checking. **C—semantic evidence persistence:** performs revision evolution with its existing evidence store, revalidation, equality certificates and guarded transfers; it is evaluated without algorithmic optimization for this experiment.

All arms use matching inputs, a fixed base descriptor, offline operation, the same Z3 setup, and the same nine obligations. Condition A sees the destination; B and C also see the source and setup-phase artifacts. Initial setup is measured separately from the primary phase-1 outcome. C and B intentionally differ in the representations and algorithms used; therefore the experiment tests these implemented conditions, not an isolated causal effect of one data structure.

3.4 Primary outcome, controls and audit

The primary metric $N_X = \sum_{i=1}^{64} n_X(i)$ counts effective invocations of the underlying proving routine in phase 1, one solver check per invocation, after cache hits. A separate counter over `z3.Solver.check` checks this accounting. The experiment prescribes five repeated runs

Table 1: Actual phase-1 solver calls in EXP-019. Lower is better.

Scope	A	B	C
64 transitions	576	96	252
61 non-flagged transitions	549	86	233

per condition/transition, fixed seeded ordering, and consistency of solver counts and verdicts between repetitions. Runtime, CPU, setup cost, memory and storage are distinct from the pre-specified primary metric; solver-call savings cannot be interpreted as equivalent wall-clock savings.

The validity criterion for a reused certificate checks that subjects refer to the correct revisions, causal closure belongs to the destination, and the relevant tool base and transfer premises are current. Stale evidence counts as contractually invalid even when its conclusion remains accidentally true. Semantic conclusions are separately checked against an oracle and interpreter. Eight adversarial controls probe irrelevant note/doc changes, misleading labels, alternative valid certificates and intentionally malformed certificate content or base descriptors. The recorded evaluation reports a separate internal audit by an agent designated Codex Sol, including recomputation from underlying cells and one complete repeated execution. This is a valuable procedural check but *not* independent peer review by an unaffiliated researcher.

4 Results

4.1 Safety and completeness (R1)

All 576 target obligations were decided in the reported evaluation; the recorded outcome also includes 64 equivalence checks and 48 refutations. The eight controls passed. No confirmed contractual reuse violation, semantic contradiction, undecided result or calibration failure was reported. Consequently R1 was sustained *within this domain and its tested controls*; this is not evidence of soundness for arbitrary languages, long histories or changed tool bases.

4.2 Full recomputation and memoization (R2–R3)

Table 1 gives the primary totals. Relative to A, the store uses

$$\rho_A = \frac{252}{576} = 0.4375, \tag{1}$$

which is a 56.25% reduction in solver calls. Relative to the stronger comparator B,

$$\rho_B = \frac{252}{96} = 2.625, \tag{2}$$

so C incurs 156 additional checks, or a 162.5% increase relative to B. Restricting the comparison to 61 non-flagged transitions yields $233/86 \approx 2.709$, preserving the qualitative result. R2 was sustained and the hypothesized overall superiority in R3 was refuted for this implementation.

Table 2: Reported phase-1 calls by change stratum, B versus C.

Stratum	B	C
Unrelated	0	56
Equivalent	40	32
Non-equivalent	40	128
Dependency	16	36
Total	96	252

Table 2 reveals a localized advantage: C uses eight fewer calls than B in equivalent revisions. This does not reverse the global comparison. The memoization comparator's observed total of 96 also improved upon a pre-execution prediction of 116. The recorded explanation is that Z3's `sexpr()` may serialize associatively different ASTs identically; the prediction's converse assumption about textual identity did not hold. This discrepancy was retained rather than adjusting the test post hoc.

4.3 Interpretation of the negative result

The measured outcome does not mean that persistence is incorrect or useless. It shows that, under the tested workload and primary cost definition, conventional caching avoids more solver work than the implemented store. Neither the question "can evidence be safely reused?" nor the question "does persistence reduce work compared to no reuse?" substitutes for the comparative question "does it outperform a strong incremental baseline?"

5 Post-Experiment Exploratory Follow-up

After EXP-019 was closed, hybrid prototypes P0 and P1 were built on *already known* cases: the 72 evaluation entries and 40 preparation entries, now all treated as development data. Thus their measurements cannot confirm a new preregistered claim. P0 strengthened a conventional cache with affine normalization and typed decisions; P1 added a selective mechanism for reusing evidence under residual claims.

The exploration reports 22 activated entries and a reduction from 72 to 44 phase-1 calls on the preparation subset, but paired wall-clock measurements across the known 112-entry set found $\Delta T = T_{P1} - T_{P0} = +0.514$ seconds per entry (reported bootstrap 95% interval [0.510, 0.519]). P1 used more memory and stored state. Subsequent internal audit identified three additional defects: incomplete causal versioning of certificate bases, a non-atomic publication failure path, and redundant reattempts under exhausted budgets. The source record also cautions that P0 and P1 were not a pure matched ablation. These exploratory observations illustrate why

fewer solver calls need not imply lower total cost or stronger end-to-end correctness. They are not pooled with the confirmatory EXP-019 results.

6 Threats to Validity

Construct validity. Solver invocations are a precise proxy for one resource, not engineering cost, latency, human effort or proof quality. The current paper reports no aggregate confirmatory timing effect. An always-true well-definedness condition and tiny fixed-width formulas create a ceiling effect. **Internal validity.** The experimental arms share inputs and measurement instrumentation, but C follows its own revision order, unlike A and B, and carries a richer certificate workflow. An arm-level comparison cannot isolate a single mechanism. The original B call prediction was inaccurate; the discrepancy is disclosed. **External validity.** Programs are synthetic, changes small, the base fixed, and widths restricted to 1–4 bits. Property changes, persisted refutations, generated C, realistic dependency graphs, long evolution histories, and heterogeneous solvers were not evaluated. **Reliability.** The design uses frozen protocols, hashes, audit evidence and repetition; however, this manuscript has not itself rerun the original artifacts, and its numbers were transcribed from the experiment's recorded results. Agent-conducted auditing is not equivalent to independent institutional replication. **Integrity defects and limitations.** The project records known defects of the evaluated store, none of which is corrected in the frozen implementation. D10 (evidence classified by textual phrases and position rather than typed fields) was not tested by EXP-019. D11 (a misleading identity error that leaves in-memory state modified when a dependency has no current revision) was avoided by construction and remains open. Two further defects were registered after the experiment. D12: each query for current evidence scans all retained certificates and revalidates matching ones without memory across calls, so per-query cost grows with history; the accumulated cost is quadratic or worse only under specific growth assumptions, which the 64 short transitions of EXP-019 do not exercise. D13: the store publishes its object files and then four indices with plain writes, without temporary files, atomic rename or `fsync`, so an interrupted write can leave inconsistent indices; the EXP-019 monitor recorded no partial publication, but no result relies on the store's atomicity. Finally, as an architectural limitation rather than a defect, the leaves of a certificate are attestations of the producing tool, not portable proofs that a third party can check without trusting it or redoing the verification; the evaluated contract declares them part of its trusted base. The later P1 audit exposed analogous versioning and atomicity issues in the prototype.

7 Reproducibility and Data Availability

The frozen EXP-019 instrument and protocol, the raw evaluation data, the internal audit, the result reports and this manuscript are archived at Zenodo, DOI 10.5281/zenodo.23873678, which resolves to the latest version (releases of `https://github.com/venture-synaptic/efch-exp019-artifacts`; the data and code are identical across versions `v1.0-exp019` and `v1.1-exp019`, which differ only in documentation and this manuscript). Code is licensed under Apache-2.0 and data and documents under CC-BY-4.0. The archive is an unmodified export from the private research repository: the instrument and protocol come from the tags `exp019-instrumento` (`4aa18a7`) and `exp019-protocolo` (`d94471a`), and the raw data from commit `531c847f`. The raw data are packaged as `data/exp019-resultados.tar.gz` (SHA-256 `6f7156051f73c8b6d3c336b33981961d466c31235fc83e3d04bb8f01b4e5b719`); inside it, the original evaluation digests are preserved, including the execution log (SHA-256 prefix `59c354f3`) and the analysis (prefix `82e684e7`). A file `SHA256SUMS` covers every file in the archive. Because the evaluation set is now public, rerunning it constitutes an informed reproduction rather than a new preregistered test.

8 Research Ethics, Authorship and AI Disclosure

Erik Fisher Cueva Huari is listed as the responsible human author. The source records describe use of Claude Opus to implement and coordinate portions of the instrument and prototypes, and Codex Sol for internal technical auditing, under the author's direction. AI systems are not listed as coauthors. The human author must check all factual statements, analysis, citations and source provenance before submission. This manuscript was itself drafted with AI assistance and has not yet received independent human subject-matter review. No external funding is claimed in this draft; funding and competing-interest disclosures require author confirmation.

9 Conclusion

In a pre-specified 64-transition synthetic experiment, the EFCH evidence store reused verification evidence without observed invalid transfers in the tested domain and reduced actual solver invocations by 56.25% against full reverification. However, it required 2.625 times the calls of strengthened conventional memoization and only outperformed that cache on the equivalent-change subset. This is a bounded, negative finding on differential efficiency—not a refutation of semantic persistence in general. Together with the exploratory follow-up and a subsequent conceptual review, this led the project to record the following conclusion: no differential architec-

tural advantage was identified for the evaluated evidence persistence and reuse mechanisms over incremental comparators with equivalent guarantees and optimizations; the investigation is closed within this scope, without claiming general impossibility. Future claims should be tested against comparators with equivalent guarantees and optimizations and should include end-to-end cost, changed tool bases, realistic histories and independent replication.

Acknowledgments

Acknowledgments and any institutional permissions should be confirmed by the human author before release.

References

[1] U. A. Acar, G. E. Blelloch, M. Blume, and K. Tangwongsan. An experimental analysis of self-adjusting computation. In *Proceedings of PLDI*, 2006.

[2] L. de Moura and N. Bjørner. Z3: An efficient SMT solver. In *TACAS*, pp. 337–340, 2008. DOI: 10.1007/978-3-540-78800-3_24.

[3] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén. *Experimentation in Software Engineering*. Springer, 2012. DOI: 10.1007/978-3-642-29044-2.

[4] A. Mokhov, N. Mitchell, and S. Peyton Jones. Build systems à la carte. *Proc. ACM Program. Lang.*, 2(ICFP), 2018.

[5] K. R. M. Leino and V. Wüstholz. Fine-grained caching of verification results. In *CAV*, 2015.

[6] Why3 documentation, version 1.9.0: *Getting Started* (proof sessions) and *The Why3 Tools* (replay). https://why3.org/doc/.

[7] Frama-C. *WP Plug-in Manual*, section "Proof Session & Cache". https://www.frama-c.com/download/frama-c-wp-manual.pdf.

[8] F* team. Caching verified modules. F* wiki, 2018. https://github.com/FStarLang/FStar/wiki/Caching-verified-modules.

[9] B. Beckert and V. Klebanov. Proof reuse for deductive program verification. In *SEFM*, pp. 77–86, 2004. DOI: 10.1109/SEFM.2004.33.

[10] C. Cadar, D. Dunbar, and D. Engler. KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs. In *OSDI*, 2008.

[11] W. Visser, J. Geldenhuys, and M. B. Dwyer. Green: Reducing, reusing and recycling constraints in program analysis. In *FSE*, 2012.

[12] B. Godlin and O. Strichman. Regression verification. In *DAC*, 2009.

[13] Microsoft Research. SymDiff: Differential program verifier. https://www.microsoft.com/en-us/research/project/symdiff-differential-program-verifier/.

[14] G. Fedyukovich, O. Sery, and N. Sharygina. eVolCheck: Incremental upgrade checker for C. In *TACAS*, LNCS 7795, pp. 292–307, 2013.

[15] S. Asadi, M. Blicha, A. E. J. Hyvärinen, G. Fedyukovich, and N. Sharygina. SMT-based verification of program changes through summary repair. *Formal Methods in System Design*, 60:350–380, 2023. DOI: 10.1007/s10703-023-00423-0.

[16] P. Schrammel, D. Kroening, M. Brain, R. Martins, T. Teige, and T. Bienmüller. Incremental bounded model checking for embedded software (extended version). arXiv:1409.5872, 2014.

[17] K. Fazekas, F. Pollitt, M. Fleury, and A. Biere. Incremental proofs for bounded model checking. In *MBMV*, 2024.

[18] G. C. Necula. Proof-carrying code. In *POPL*, 1997.

[19] B. Ekici et al. SMTCoq: A plug-in for integrating SMT solvers into Coq. In *CAV*, 2017.